# CP312 Assignment 1

I certify that this Assignment is entirely my own work and that all External resources, if used, have been properly Acknowledged.

1.

a) The outer loop pattern runs for $i = 1, 2, 4, \ldots$ (powers of 2). The inner loop runs $i$ times.

Total steps $= \sum_{k=0}^{\log_2 n} 2^k \rightarrow 2^{\log_2 n + 1} - 1 = 2(n) - 1$

Using the geometric series formula, this equals $2n - 1$. Therefore, $T(n) \in \Theta(n)$

b) The loop initializes at $i = n$ and divides $i$ by 6 in each iteration ($i = \frac{i}{6}$). The loop continues until $i$ is no longer greater than 0. To find the number of iterations, we would need to solve for when the value reaches 1:

$$\frac{n}{6^k} = 1$$

$$6^k = n$$

$$k = \log_6(n)$$

Since the operations inside the loop are constant time ($O(1)$), the total time complexity is determined by the # of iterations. Therefore, $T(n) \in \Theta(\log n)$

c) The outer loop runs from $i = 1$ to $n$. The inner loop starts at $j = 1$ and multiplies $j$ by 3 until $j \geq i$. This means the inner loop runs $\log_3 i$ times for each iteration of the outer loop. The total complexity is the sum of the inner loop costs:

$$T(n) = \sum_{i=1}^{n} \log_3 i$$

$$T(n) = \log_3(1 \cdot 2 \cdot 3 \cdot \ldots \cdot n) = \log_3(n!) \rightarrow \log a + \log b = \log ab$$

Using Sterlings Approximation, we know that $\log(n!) \approx n \log n$. Therefore, $T(n) \in \Theta(n \log n)$

2.

a) Yes, insertion sort can be implemented on linked lists with the same $O(n^2)$ time complexity as the array-based version. But in the Array based implementation, the cost is split between comparisions $O(n)$ & shifting elements $O(n)$. For linked list implementation
↳ For insertion cost, once the correct position is found, inserting a node only requires updating pointers which is $O(1)$
↳ For comparison cost we must traverse the list sequentially from the start to find the correct insertion point. For the worst case the traversal takes $O(n)$ comparisions for each element.

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

∴ Time complexity is $O(n^2)$

b) Algorithim OrderPlayers(Players[1...n]):
Initialize List L containing only Players[1]

For i from 2 to n:
Let NewPlayer = Players[i]

If NewPlayer did not lose to L.head:
Prepend NewPlayer to L

Else If L.tail did not lose to NewPlayer:
Append NewPlayer to L

Else: Iterate through L to find node "current" such that:
(current did not lose to NewPlayer) AND
(NewPlayer did not lose to current.next)
Insert NewPlayer after 'current'

Return L

2.

b) Explanation: I solved this using an incremental approach similar to insertion sort. I maintained a list of players that is valid according to the condition. For each new player added:

↳ If new player won/drew against first player I inserted them at the front.

↳ If Last player won/drew against new player I inserted New player at the end.

↳ otherwise First beat New ↳ New beat last I inserted New between $P_i$ ('beat new) ↳ $P_{i+1}$ (new beat).

For the time complexity for the worst case we need to scan the existing list to find the correct insertion point.

↳ When inserting the 2nd player we do 1 comparision

↳ When inserting the $k^{th}$ player we do upto k-1 comparisions

$$T(n) = \sum_{k=2}^{n} (k-1) = 1+2+\dots+(n-1) = \frac{n(n-1)}{2}$$

∴ The Time complexity is $O(n^2)$

3.

Prove $4^n \notin O(2^n)$:

Assume $4^n \in O(2^n)$

Big O. Definition ↳ constraints: $c>0$ ↳ $n_0 \geq 0$ for all $n \geq n_0$

$4^n \leq c \cdot 2^n$

$\frac{4^n}{2^n} \leq c$

$2^n \leq c$

Since $2^n$ grows to infinity as $n \to \infty$, it cannot be bounded by a constant. This is an contradiction. Therefore $4^n \notin O(2^n)$

3.

Prove $a^{\log_b x} = x^{\log_b a}$:

Let $y = a^{\log_b x}$ (Take log of both sides)

$\log_b y = \log_b (a^{\log_b x})$ (use power rule $\log n^r = r \log n$

$\log_b y = (\log_b x) \cdot (\log_b a)$ (swap terms)

$\log_b y = (\log_b a) \cdot (\log_b x)$ (use power rule in reverse)

$\log_b y = \log_b (x^{\log_b a})$

$y = x^{\log_b a}$

$\therefore a^{\log_b x} = x^{\log_b a}$

Prove $\lg(n!) \in \Theta(n \lg n)$:

upper bound ($O$):

$n! = 1 \cdot 2 \cdot \ldots \cdot n \leq n \cdot n \cdot \ldots \cdot n = n^n$ (take log of both sides)

$\lg(n!) \leq \lg(n^n) = n \lg n$

So, $\lg(n!) \in O(n \lg n)$

Lower bound ($\Omega$):

$n! = 1 \cdot \ldots \cdot \frac{n}{2} \cdot \ldots \cdot n \geq (\frac{n}{2})^{n/2}$ (take log of both sides)

$\lg(n!) \geq \lg((\frac{n}{2})^{n/2})$

$\lg(n!) \geq \frac{n}{2} \lg(\frac{n}{2}) = \frac{n}{2}(\lg n - 1)$

Since $\frac{n}{2}(\lg n - 1)$ grows at the rate of $n \lg n$,

$\lg(n!) \in \Omega(n \lg n)$

Since $\lg(n!) \in O(n \lg n)$ and $\lg(n!) \in \Omega(n \lg n)$, $\lg(n!) \in \Theta(n \lg n)$. I know my proof is correct because I showed that $\lg(n!)$ is both $O(n \lg n)$ and $\Omega(n \lg n)$

4.

$g_1 = (n+1)!$

$g_2 = n!$

$g_3 = (\lg n)^{\lg n}$ $(= n^{\lg \lg n})$

$g_4 = n^{\lg \lg n}$ (same as above)

$g_5 = (\lg n)!$

$g_6 = n^{100}$

$g_7 = 4^{\lg n}$ $(= (2^2)^{\lg n} = 2^{\lg n^2} = n^2)$

$g_8 = n^2$ (same as above)

$g_9 =$ [redacted] $2^{\lg n}$ (same as below)

$g_{10} = 2^{\lg n}$ (Equal to $n$)

$g_{11} =$ [redacted] $\frac{n}{100}$ $(O(n))$

$g_{12} = \ln n$

$g_{13} = \sqrt{\lg n}$

$g_{14} = \ln \ln n$

$g_{15} = 1$ (constant)